Formal Methods and Specification—Assignment 3

From now on, when proving some predicate-logical formula, you may use equivalence rules (see Section 1.5 of the lecture notes or lecture 2 of the course MI-TES). You may also assume the axioms of all necessary theories. Moreover, in addition to the Peano axioms, you may assume all facts that intuitively hold for the natural numbers and integers (e.g., associativity, commutativity of addition and multiplication) and all relevant definitions (e.g., the definition of division of natural numbers). Also, you may use the proof rules liberally. That is, you do not have to follow the proof rules in all details, as long as you are aware of how the missing details would look like.

Exercise 1

In the following, the variable name a refers to arrays, i, j, k refer to non-negative integers. The variables x, y refer to array elements, which you may assume to be integers. For all other variable names, and any constant that you use, you can assign types as you want. You can find examples in Section 2.3 of the lecture notes (the examples might give you important hints).

(a) In the theory of arrays, prove

$$\forall x \exists a \,.\, a[0] = x. \qquad (\searrow)$$

(b) In the theory of arrays, prove

$$\forall i \exists a \,.\, a[i] = 2a[i+1] \qquad \text{(1.5 points)}$$

(c) In the theory of arrays, prove

$$\exists a, i, x \,.\, \mathsf{write}(a, i, x)[0] = x + 3. \qquad \text{(1.5 points)}$$

Throughout you may use any knowledge about the integers you want.

Exercise 2

Convert each of the following basic paths into SSA, and then write down its verification condition. Check whether the verification condition holds. If it holds, write down a proof (that can be short and informal). If the verification condition does not hold, find a counter-example, that is, a variable assignment that both shows that the formula does not hold, and at the same time represents an initial state for which execution of the basic path results in a bug. For each of those steps, there are examples on the slides of the lecture "Correctness of Programs Without Control Structures". Especially, there is also a slide that explains how to handle array assignments. All program variables range over the integers.

(a) **assume** $x \geq 0$; $y \leftarrow x$; $z \leftarrow y$; @ $z \geq 0$

(b) **assume** $x \geq 0$; $z \leftarrow y$; $y \leftarrow x$; @ $z \geq 0$

(c) $x \leftarrow x - k$; **assume** $k \leq x$; @ $x \geq 0$

(d) **assume** $x \geq 0$; $y \leftarrow x$; **input** x; @ $x \geq 0$

(e) $y \leftarrow x$; **input** x; **assume** $x \geq 0$; @ $y \geq 0$

(f) $a[i] \leftarrow 7$; $i \leftarrow i + 1$; $a[i] \leftarrow 8$; @ $i \geq a[i]$

(6×0.5 point)

Exercise 3

Consider the following program (all variables range over the natural numbers including zero):

$x_0 \leftarrow x$
$i \leftarrow 0$
while $i < n$ **do**
    @ $x = x_0 + \sum_{k=0}^{i-1} a[k]$
    $x \leftarrow x + a[i]$
    $i \leftarrow i + 1$
@ $x = x_0 + \sum_{k=0}^{n-1} a[k]$

(a) Write down all basic paths of this program. Take care that the basic paths that begin at the loop invariant start with a corresponding **assume** statement.

(1.5 points)

(b) For each basic path, write down the corresponding verification condition.

(1.5 points)

(c) For each verification condition, check whether it holds. It suffices to do an informal check, so no proof is needed, here. (1 point)

(d) If any verification condition does not hold, modify the assertion in the loop such that the verification conditions hold. However, do not change the assertion in the last program line. (1 point)

Mark exercise 9d only, if you do find a verification condition that does not hold, and a modification of the assertion that makes all verification conditions hold.

Exercise 4

Consider the following program (all variables range over the integers):

assume $n \neq 0$
$k \leftarrow n$
if $n < 0$ **then**
    $l \leftarrow 1$
else
    $l \leftarrow -1$
for $i \leftarrow 0$ **to** $|n| - 1$ **do**
    @ $k = n + li \wedge l = -sgn(n)$
    $k \leftarrow k + l$
    @ $k = n + l(i + 1) \wedge l = -sgn(n)$
@ $k = 0$

(a) Write down all basic paths of this program. Take care that the basic paths that begin at the loop invariant start with a corresponding **assume** statement.

(1.5 points)

(b) For every basic path, check whether it is correct. If you are able to explain your answer without verification conditions, you do not need to write them down. But, of course, you are allowed to use them, if you want. (1 point)